

Practical No. 1 — Data Handling

Aim

To perform various data handling operations using Python, including:

1. Parsing text, CSV, HTML, XML and JSON documents and extracting relevant data while identifying missing values and anomalies.
2. Reading and writing binary files.
3. Searching, splitting and replacing strings using regular expressions.
4. Designing and populating a relational database and performing CRUD operations using SQL.

1. Parsing text, CSV, HTML, XML and JSON documents and extracting relevant data while identifying missing values and anomalies.

Practical No. 1 — Data Parsing, File Handling, Regular Expressions and SQL CRUD

Exercise 1 :

```
import pandas as pd
import re
import json
import csv
import sqlite3
import os
import xml.etree.ElementTree as ET
from bs4 import BeautifulSoup

print("All required libraries imported successfully!")
```

All required libraries imported successfully!

STEP 1 — Create a Text File

```
text_data = """Name: Rahul
Age: 21
City: Mumbai
Course: Data Science
"""

with open("student.txt", "w") as file:
    file.write(text_data)

print("Text file created successfully!")
```

Text file created successfully!

STEP 2 — Read and Parse the Text File

```
with open("student.txt", "r") as file:
    text = file.read()

print(text)
```

Name: Rahul
Age: 21
City: Mumbai
Course: Data Science

STEP 3 — Extract Data from Text File

```
student_data = {}

for line in text.splitlines():
    key, value = line.split(":", 1)
    student_data[key.strip()] = value.strip()

print(student_data)

{'Name': 'Rahul', 'Age': '21', 'City': 'Mumbai', 'Course': 'Data Science'}
```

STEP 4 — Create a CSV File

```
students = [
    [1, "Rahul", 21, "Mumbai", "Data Science"],
    [2, "Sneha", 20, "Pune", "Computer Science"]]
```

```

[3, "Amit", None, "Nashik", "Data Science"],
[4, "Priya", 22, None, "AI"]
]

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([
        "Student_ID",
        "Name",
        "Age",
        "City",
        "Course"
    ])

    writer.writerows(students)

print("CSV file created successfully!")

```

CSV file created successfully!

STEP 5 — Read the CSV File

```

df = pd.read_csv("students.csv")
display(df)

```

	Student_ID	Name	Age	City	Course
0	1	Rahul	21.0	Mumbai	Data Science
1	2	Sneha	20.0	Pune	Computer Science
2	3	Amit	NaN	Nashik	Data Science
3	4	Priya	22.0	NaN	AI

STEP 6 — Check Missing Values

```

print("Missing values:")
print(df.isnull().sum())

```

```

Missing values:
Student_ID    0
Name          0
Age           1
City          1
Course        0
dtype: int64

```

STEP 7 — Check Anomalies

```

invalid_age = df[
    (df["Age"].notnull()) &
    (df["Age"] <= 0)
]

```

```

print("Invalid age records:")
display(invalid_age)

```

```

Invalid age records:
  Student_ID Name Age City Course

```

```

duplicate_ids = df[
    df["Student_ID"].duplicated()
]
print("Duplicate Student IDs:")
display(duplicate_ids)

```

```

Duplicate Student IDs:
  Student_ID Name Age City Course

```

STEP 8 — Create an HTML File

```

html_data = """
<html>
<body>
<h1>Student Information</h1>

<table>
<tr>
<th>ID</th>
<th>Name</th>
<th>Course</th>
</tr>

<tr>
<td>1</td>
<td>Rahul</td>
<td>Data Science</td>
</tr>

<tr>
<td>2</td>
<td>Sneha</td>
<td>Computer Science</td>
</tr>
</table>

```

```

</body>
</html>
"""

with open("students.html", "w") as file:
    file.write(html_data)

print("HTML file created successfully!")

```

HTML file created successfully!

STEP 9 — Parse HTML

```

with open("students.html", "r") as file:
    html = file.read()

soup = BeautifulSoup(html, "html.parser")

table = soup.find("table")

rows = []

for row in table.find_all("tr"):
    cells = row.find_all(["th", "td"])
    rows.append([
        cell.get_text(strip=True)
        for cell in cells
    ])

for row in rows:
    print(row)

```

```

['ID', 'Name', 'Course']
['1', 'Rahul', 'Data Science']
['2', 'Sneha', 'Computer Science']

```

STEP 10 — Create an XML File

```

xml_data = """
<students>
  <student>
    <id>1</id>
    <name>Rahul</name>
    <course>Data Science</course>
  </student>

  <student>
    <id>2</id>
    <name>Sneha</name>
    <course>Computer Science</course>
  </student>
</students>
"""

with open("students.xml", "w") as file:
    file.write(xml_data)

print("XML file created successfully!")

```

XML file created successfully!

STEP 11 — Parse XML

```

tree = ET.parse("students.xml")
root = tree.getroot()

for student in root.findall("student"):
    student_id = student.find("id").text
    name = student.find("name").text
    course = student.find("course").text

    print(
        "ID: ", student_id,
        "| Name: ", name,
        "| Course: ", course
    )

```

```

ID: 1 | Name: Rahul | Course: Data Science
ID: 2 | Name: Sneha | Course: Computer Science

```

STEP 12 — Create a JSON File

```

json_data = {
    "students": [
        {
            "id": 1,
            "name": "Rahul",
            "course": "Data Science"
        },
        {
            "id": 2,
            "name": "Sneha",
            "course": "Computer Science"
        }
    ]
}

with open("students.json", "w") as file:
    json.dump(json_data, file, indent=4)

print("JSON file created successfully!")

```

JSON file created successfully!

STEP 13 — Parse JSON

```
with open("students.json", "r") as file:
    data = json.load(file)

for student in data["students"]:
    print(
        "ID:", student["id"],
        "| Name:", student["name"],
        "| Course:", student["course"]
    )
```

```
ID: 1 | Name: Rahul | Course: Data Science
ID: 2 | Name: Sneha | Course: Computer Science
```

2. Reading and writing binary files.

Exercise 2 — Read and Write Binary Files

STEP 14 — Write Data to a Binary File

```
import pickle

print("Pickle imported successfully!")

Pickle imported successfully!
```

STEP 15 — Create Data

```
student = {
    "Student_ID": 101,
    "Name": "Rahul",
    "Age": 21,
    "Course": "Data Science"
}

print(student)

{'Student_ID': 101, 'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science'}
```

STEP 16 — Write the Data to a Binary File

```
with open("student.dat", "wb") as file:
    pickle.dump(student, file)

print("Data written to binary file successfully!")

Data written to binary file successfully!
```

STEP 17 — Read the Binary File

```
with open("student.dat", "rb") as file:
    loaded_student = pickle.load(file)

print("Data read from binary file:")
print(loaded_student)

Data read from binary file:
{'Student_ID': 101, 'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science'}
```

STEP 18 — Write Multiple Records

```
students = [
    {
        "Student_ID": 101,
        "Name": "Rahul",
        "Age": 21,
        "Course": "Data Science"
    },
    {
        "Student_ID": 102,
        "Name": "Sneha",
        "Age": 20,
        "Course": "Computer Science"
    },
    {
        "Student_ID": 103,
        "Name": "Amit",
        "Age": 22.
    }
]
```

```
        "Course": "Artificial Intelligence"
    }
]

with open("students.dat", "wb") as file:
    pickle.dump(students, file)

print("Multiple student records written successfully!")
```

Multiple student records written successfully!

STEP 19 — Read Multiple Records

```
with open("students.dat", "rb") as file:
    loaded_students = pickle.load(file)

print("Students from binary file:")

for student in loaded_students:
    print(student)

Students from binary file:
{'Student_ID': 101, 'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science'}
{'Student_ID': 102, 'Name': 'Sneha', 'Age': 20, 'Course': 'Computer Science'}
{'Student_ID': 103, 'Name': 'Amit', 'Age': 22, 'Course': 'Artificial Intelligence'}
```

STEP 20 — Verify the Binary File Exists

```
import os

print("student.dat exists:", os.path.exists("student.dat"))
print("students.dat exists:", os.path.exists("students.dat"))
```

student.dat exists: True
students.dat exists: True

Exercise 3: Searching, Splitting and Replacing Strings using Regular Expressions

STEP 21 — Import the re Module

```
import re

print("Regular expression module imported successfully!")
```

Regular expression module imported successfully!

STEP 22 — Searching Using Regular Expression

```
text = "My email is rahul@gmail.com and my phone number is 9876543210."

print(text)

My email is rahul@gmail.com and my phone number is 9876543210.

pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"

result = re.search(pattern, text)

if result:
    print("Email found:", result.group())
else:
    print("Email not found")
```

Email found: rahul@gmail.com

STEP 23 — Search for a Phone Number

```
phone_pattern = r"\b\d{10}\b"

phone = re.search(phone_pattern, text)

if phone:
    print("Phone number found:", phone.group())
else:
    print("Phone number not found")

Phone number found: 9876543210
```

STEP 24 — Find All Matching Values

```
text = """
Contact rahul@gmail.com
or sneha@gmail.com
or amit@yahoo.com
"""

emails = re.findall(
    r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
    text
)

print("Emails found:")
print(emails)
```

```
Emails found:
['rahul@gmail.com', 'sneha@gmail.com', 'amit@yahoo.com']
```

STEP 25 — Splitting Strings Using Regex

```
text = "Apple,Orange;Banana Mango|Grapes"
print(text)
```

Apple,Orange;Banana Mango|Grapes

```
parts = re.split(
    r"[;, |]+",
    text
)

print(parts)
```

['Apple', 'Orange', 'Banana', 'Mango', 'Grapes']

STEP 26 — Another Splitting Example

```
sentence = "Python is easy to learn"
words = re.split(
    r"\s+",
    sentence
)

print(words)
```

['Python', 'is', 'easy', 'to', 'learn']

STEP 27 — Replacing Strings Using Regex

```
text = "Contact Rahul at 9876543210."
new_text = re.sub(
    r"\b\d{10}\b",
    "XXXXXXXXXX",
    text
)

print(new_text)
```

Contact Rahul at XXXXXXXXXX.

STEP 28 — Replace Multiple Numbers

```
text = "Rahul: 9876543210, Sneha: 9123456789"
new_text = re.sub(
    r"\b\d{10}\b",
    "XXXXXXXXXX",
    text
)

print(new_text)
```

Rahul: XXXXXXXXXX, Sneha: XXXXXXXXXX

STEP 29 — Replace Extra Spaces

```
text = "Python    is    very    easy"
clean_text = re.sub(
    r"\s+",
    " ",
    text
)

print(clean_text)
```

Python is very easy

Exercise 4: SQL CRUD Operations

```
import sqlite3

print("SQLite imported successfully!")
```

SQLite imported successfully!

STEP 30 — Create the Database

```
connection = sqlite3.connect("student_database.db")
print("Database created successfully!")
```

Database created successfully!

STEP 31 — Create a Student Table

```
cursor = connection.cursor()
```

```

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    Student_ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Age INTEGER,
    City TEXT,
    Course TEXT
)
""")

connection.commit()

print("Students table created successfully!")

```

Students table created successfully!

STEP 32 — CREATE: Insert Records

```

students = [
    (1, "Rahul", 21, "Mumbai", "Data Science"),
    (2, "Sneha", 20, "Pune", "Computer Science"),
    (3, "Amit", 22, "Nashik", "Artificial Intelligence"),
    (4, "Priya", 21, "Thane", "Data Science")
]

cursor.executemany("""
INSERT INTO students
(Student_ID, Name, Age, City, Course)
VALUES (?, ?, ?, ?, ?)
""", students)

connection.commit()

print("Student records inserted successfully!")

```

Student records inserted successfully!

STEP 33 — READ: Display All Records

```

cursor.execute("SELECT * FROM students")

records = cursor.fetchall()

for record in records:
    print(record)

```

```

(1, 'Rahul', 21, 'Mumbai', 'Data Science')
(2, 'Sneha', 20, 'Pune', 'Computer Science')
(3, 'Amit', 22, 'Nashik', 'Artificial Intelligence')
(4, 'Priya', 21, 'Thane', 'Data Science')

```

STEP 34 — READ: Select Specific Columns

```

▶ cursor.execute("""
SELECT Name, Course
FROM students
""")

records = cursor.fetchall()

for record in records:
    print(record)

```

```

... ('Rahul', 'Data Science')
    ('Sneha', 'Computer Science')
    ('Amit', 'Artificial Intelligence')
    ('Priya', 'Data Science')

```

STEP 35 — READ: Search for a Student

```

cursor.execute("""
SELECT *
FROM students
WHERE Course = 'Data Science'
""")

records = cursor.fetchall()

for record in records:
    print(record)

```

```

(1, 'Rahul', 21, 'Mumbai', 'Data Science')
(4, 'Priya', 21, 'Thane', 'Data Science')

```

STEP 36 — UPDATE: Modify a Record

```

cursor.execute("""
UPDATE students
SET City = 'Pune'
WHERE Student_ID = 1
""")

connection.commit()

print("Student record updated successfully!")

```

Student record updated successfully!

STEP 37— Verify the UPDATE

```
cursor.execute("""
SELECT *
FROM students
WHERE Student_ID = 1
""")

print(cursor.fetchone())

(1, 'Rahul', 21, 'Pune', 'Data Science')
```

STEP 38 — DELETE: Remove a Record

```
cursor.execute("""
DELETE FROM students
WHERE Student_ID = 4
""")

connection.commit()

print("Student record deleted successfully!")

Student record deleted successfully!
```

STEP 39 — Verify DELETE

```
cursor.execute("SELECT * FROM students")

records = cursor.fetchall()

for record in records:
    print(record)

(1, 'Rahul', 21, 'Pune', 'Data Science')
(2, 'Sneha', 20, 'Pune', 'Computer Science')
(3, 'Amit', 22, 'Nashik', 'Artificial Intelligence')
```

STEP 40 — Check Number of Records

```
cursor.execute("SELECT COUNT(*) FROM students")

count = cursor.fetchone()[0]

print("Total students:", count)

Total students: 3
```

STEP 41 — Close the Database

```
connection.close()

print("Database connection closed.")

Database connection closed.
```

```
print("Mayuri Narsu Patil")

Mayuri Narsu Patil
```

Practical No. 2

Aim: Perform the Extraction Transformation and Loading (ETL) process to construct the database in the Sqlserver / Power BI.

The three stages of ETL are:

1. Extraction

Data is collected from one or more data sources such as Excel files, CSV files, databases or APIs.

2. Transformation

The extracted data is cleaned and modified according to analytical requirements. Common transformations include:

- Removing duplicate records
- Handling missing values
- Changing data types
- Renaming columns
- Filtering records
- Creating calculated columns
- Categorizing data

3. Loading

The transformed data is loaded into the target system or data model where it can be analyzed and visualized.

The screenshot displays the Power BI Desktop interface. The 'Get data' menu is open, showing various data sources. The 'products.csv' data source is selected, and the 'Load' button is highlighted. The 'products.csv' data is displayed in a table format with columns: ProductID, ProductName, SupplierID, CategoryID, QuantityPerUnit, UnitPrice, UnitsInStock, UnitsOnOrder, and ReorderLevel. The data is loaded into the Power Query Editor, which shows the same table with data type detection and transformation options.

ProductID	ProductName	SupplierID	CategoryID	QuantityPerUnit	UnitPrice	UnitsInStock	UnitsOnOrder	ReorderLevel
1	Chai	1	1	10 boxes x 20 bags	18	39	0	0
2	Chang	1	1	24 - 12 oz bottles	19	17	40	0
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10	13	70	0
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22	53	0	0
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35	0	0	0
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25	120	0	0
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30	15	0	0
8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	40	6	0	0
9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0
10	Ikura	4	8	12 - 200 ml jars	31	31	0	0
11	Queso Cabrales	5	4	1 kg pkg.	21	22	30	0
12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	38	86	0	0
13	Konbu	6	8	2 kg box	6	24	0	0
14	Tofu	6	7	40 - 100 g pkgs.	29.25	35	0	0
15	Genen Shouyu	6	2	24 - 250 ml bottles	15.5	39	0	0
16	Pavlova	7	3	32 - 500 g boxes	17.45	29	0	0
17	Alice Mutton	7	6	20 - 1 kg tins	39	0	0	0
18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0
19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	9.2	25	0	0
20	Sir Rodney's Marmalade	8	3	30 gift boxes	81	40	0	0

Help

Close & Apply, New Source, Recent Sources, Enter Data, Data source settings, Manage Parameters, Export results

Advanced Editor, Properties, Merge Queries, Append Queries, Combine Files, Combine

Choose Columns, Remove Columns, Keep Rows, Remove Rows, Sort, Split Column, Group By

ms("#Changed Type",{"ProductID", "ProductName", "QuantityPerUnit", "UnitsInStock"})

ProductID, QuantityPerUnit, UnitsInStock

100% Valid, 100% Valid, 100% Valid

0% Error, 0% Error, 0% Error

0% Empty, 0% Empty, 0% Empty

10 boxes x 20 bags, 39

24 - 12 oz bottles, 17

12 - 550 ml bottles, 13

48 - 6 oz jars, 53

36 boxes, 0

12 - 8 oz jars, 120

12 - 1 lb pkgs, 15

12 - 12 oz jars, 6

18 - 500 g pkgs, 29

12 - 200 ml jars, 31

Data Type: Whole Number

Decimal Number, Fixed decimal number, Whole Number, Percentage, Date/Time, Date, Time, Date/Time/Timezone, Duration, Text, True/False, Binary

Settings, PROPERTIES, Fields, Properties, APPLIED STEPS, Source, Promoted Headers, Changed Type, Removed Other Columns

Close & Apply, New Source, Recent Sources, Enter Data, Data source settings, Manage Parameters, Export results

Queries [2]

products, orders

Table.TransformColumns

OrderID, CustomerID

Valid, Error, Empty

100%, 0%, 0%

Valid, Error, Empty

10248 VINET

10249 TOMSP

10250 HANAR

10251 VICTE

10252 SUPRD

10253 HANAR

10254 CHOPS

Close & Apply, New Source, Recent Sources, Enter Data, Data source settings, Manage Parameters, Export results

Queries [3]

products, orders, order_details

Table.TransformColumns

OrderID, ProductID

Valid, Error, Empty

100%, 0%, 0%

Valid, Error, Empty

1 10248

2 10248

3 10248

4 10249

5 10249

Tools, Help

Merge

Select a table and matching columns to create a merged table.

orders

OrderID	CustomerID	EmployeeID	OrderDate	RequiredDate	ShippedDate	ShipVia	F
10248	VINET	5	04-07-1996 00:00:00	01-08-1996 00:00:00	16-07-1996 00:00:00	3	
10249	TOMSP	6	05-07-1996 00:00:00	16-08-1996 00:00:00	10-07-1996 00:00:00	1	
10250	HANAR	4	08-07-1996 00:00:00	05-08-1996 00:00:00	12-07-1996 00:00:00	2	
10251	VICTE	3	08-07-1996 00:00:00	05-08-1996 00:00:00	15-07-1996 00:00:00	1	

order_details

OrderID	ProductID	UnitPrice	Quantity	Discount
10248	11	14	12	0
10248	42	9.8	10	0
10248	72	34.8	5	0
10249	14	18.6	9	0
10249	51	42.4	40	0

Join Kind

Left Outer (all from first, matching from second)

Use fuzzy matching to perform the merge

Fuzzy matching options

The selection matches 830 of 830 rows from the first table.

OK, Cancel

Table.TransformColumns

Search Columns to Expand

Expand, Aggregate

(Select All Columns)

OrderID, ProductID, UnitPrice, Quantity, Discount

Use original column name as prefix

OK, Cancel

Query Settings

PROPERTIES

Name: orders

APPLIED STEPS

Source, Promoted Headers, Changed Type, Merged Queries

Custom Column

Add a column that is computed from the other columns.

New column name: LineTotal

Custom column formula: = [UnitPrice] * [Quantity]

Available columns: OrderID, CustomerID, EmployeeID, OrderDate, RequiredDate, ShippedDate, ShipVia

<< Insert

OK, Cancel

No syntax errors have been detected.

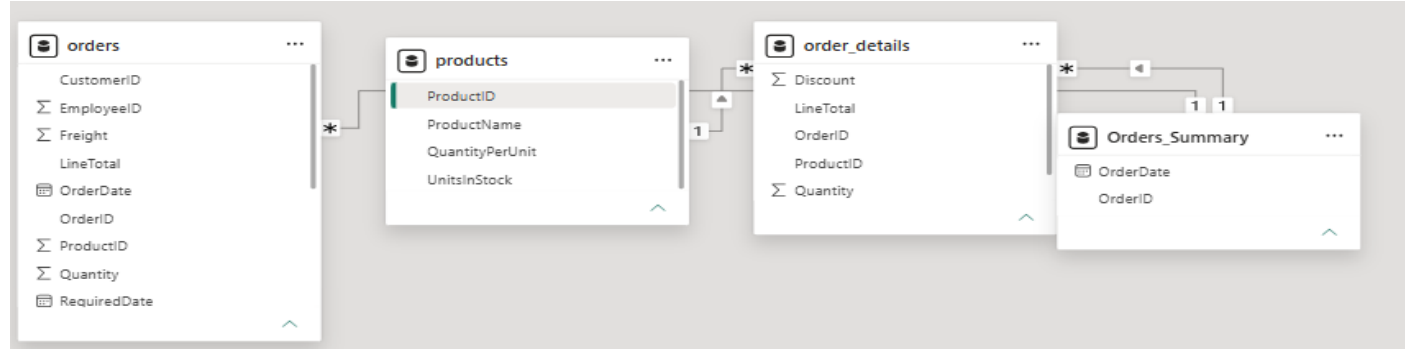
Parse	Statistics	Standard Scientific	Information	Date	Time	Duration
From Text		From Number				From Date & Time

#"Renamed Columns", "LineTotal", each [UnitPrice] * [Quantity])

123 ProductID	1.2 UnitPrice	123 Quantity	ABC 123 LineTotal
100% Valid	100% Valid	100% Valid	100% Valid
0% Error	0% Error	0% Error	0% Error
0% Empty	0% Empty	0% Empty	0% Empty
	11	14	12
	42	9.8	10
	72	34.8	5
	14	18.6	9
	51	42.4	40
	41	7.7	10
	51	42.4	35
	65	16.8	15
	22	16.8	6
	57	15.6	15
	65	16.8	20
	20	64.8	40

The screenshot shows the Power BI Desktop interface. On the left, the 'Visualizations' pane is open, displaying various chart and table icons. The 'Data' pane on the right shows a list of tables: 'order_details', 'orders', and 'products'. Below this, a 'Queries' list shows 'Products', 'Orders', 'Order_Details', and 'Orders_Summary'. The main area displays a table with columns 'PostalCode', 'ShipCountry', and 'Status'. The 'Status' column has a data card showing 100% Valid, 0% Error, and 0% Empty. The table data shows two rows with 'PostalCode' values 1 and 2, both with 'ShipCountry' as 'France'.

products	123 OrderID OrderDate	
orders	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%
order_details		
Orders_Summary		
	10248	04-07-1996 00:00:00
	10249	05-07-1996 00:00:00
	10250	08-07-1996 00:00:00
	10251	08-07-1996 00:00:00
	10252	09-07-1996 00:00:00
	10253	10-07-1996 00:00:00
	10254	11-07-1996 00:00:00
	10255	12-07-1996 00:00:00
	10256	15-07-1996 00:00:00
	10257	16-07-1996 00:00:00
	10258	17-07-1996 00:00:00
	10259	18-07-1996 00:00:00
	10260	19-07-1996 00:00:00
	10261	19-07-1996 00:00:00
	10262	22-07-1996 00:00:00
	10263	23-07-1996 00:00:00
	10264	24-07-1996 00:00:00
	10265	25-07-1996 00:00:00



Practical No.3

Aim: To perform single and multiple document insertion and retrieval using MongoDB shell (mongosh).

Step 1: Insert Single Document

Query:

```
db.items.insertOne({ name: "laptop", price: 999 });
```

Output:

```
test> use practical3
switched to db practical3
practical3> db.items.insertOne({ name: "laptop", price: 999 });
{
  acknowledged: true,
  insertedId: ObjectId('6a86676d4b503bee472af196')
}
```

Step 2: View Inserted Document

Query:

```
db.items.find();
```

Output:

```
practical3> db.items.find();
[
  {
    _id: ObjectId('6a86676d4b503bee472af196'),
    name: 'laptop',
    price: 999
  }
]
practical3> |
```

Step 3: Insert Multiple Documents

Query:

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 } ]);
```

Output:

```
practical3> db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a8669d64b503bee472af197'),
    '1': ObjectId('6a8669d64b503bee472af198'),
    '2': ObjectId('6a8669d64b503bee472af199')
  }
}
practical3> |
```

Step 4: View Products in Formatted JSON

Query:

```
db.products.find().pretty();
```

Output:

```
practical3> db.products.find().pretty();
[
  {
    _id: ObjectId('6a8669d64b503bee472af197'),
    name: 'phone',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('6a8669d64b503bee472af198'),
    name: 'tablet',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6a8669d64b503bee472af199'),
    name: 'watch',
    price: 150,
    stock: 0
  }
]
practical3> |
```

Practical no. 4

Aim

To perform noise elimination, feature selection, and exploratory data analysis (EDA) on a dataset using Pandas, NumPy, Matplotlib, and Scikit-learn.

Practical No. 4 : Python program demonstrating Noise Elimination, Feature Selection, and Exploratory Data Analysis (EDA) using Pandas, NumPy, Matplotlib, and Scikit-learn.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# -----
# Sample Dataset
# -----
data = {
    'Age': [20, 21, 22, 23, 24, 100],
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# -----
# 1. Noise Elimination (Outlier Removal)
# -----
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]

print("\nDataset after Noise Elimination:")
print(df_clean)

# -----
# 2. Feature Selection
# -----
selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selector.get_support()]

print("\nSelected Features:")
print(selected_columns)

# -----
# 3. Exploratory Data Analysis (EDA)
# -----

print("\nDataset Information:")
df_clean.info()

print("\nStatistical Summary:")
print(df_clean.describe())

print("\nCorrelation Matrix:")
print(df_clean.corr())

# Histogram
df_clean.hist(figsize=(8, 6))
plt.suptitle("Histogram of Features")
plt.show()

# Boxplot
df_clean.boxplot(figsize=(8, 5))
plt.title("Boxplot of Dataset")
plt.show()
```

```
Original Dataset:
  Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
5  100     20          50         1

Dataset after Noise Elimination:
  Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1

Selected Features:
Index(['Age', 'Marks', 'Attendance'], dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Age         5 non-null        int64
1   Marks        5 non-null        int64
2  Attendance    5 non-null        int64
3   Constant     5 non-null        int64
dtypes: int64(4)
memory usage: 200.0 bytes

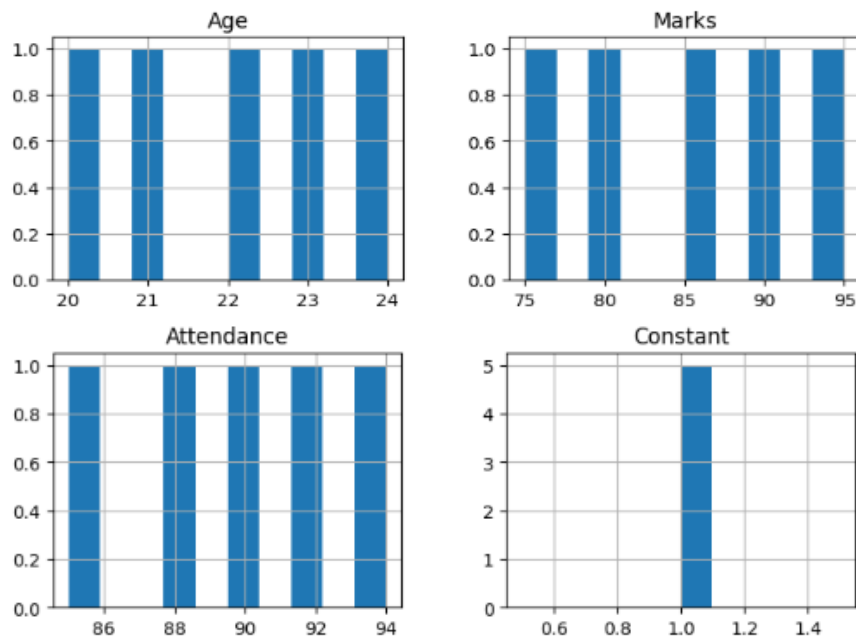
Statistical Summary:
      Age      Marks  Attendance  Constant
count  5.000000  5.000000   5.000000    5.0
mean   22.000000  85.000000   89.800000    1.0
```

std	1.581139	7.905694	3.49285	0.0
min	20.000000	75.000000	85.00000	1.0
25%	21.000000	80.000000	88.00000	1.0
50%	22.000000	85.000000	90.00000	1.0
75%	23.000000	90.000000	92.00000	1.0
max	24.000000	95.000000	94.00000	1.0

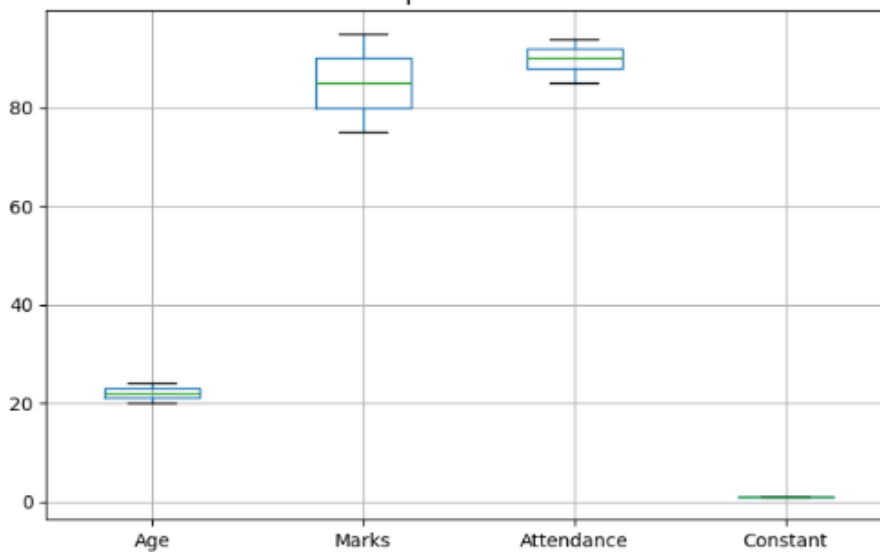
Correlation Matrix:

	Age	Marks	Attendance	Constant
Age	1.000000	1.000000	0.995893	NaN
Marks	1.000000	1.000000	0.995893	NaN
Attendance	0.995893	0.995893	1.000000	NaN
Constant	NaN	NaN	NaN	NaN

Histogram of Features



Boxplot of Dataset



```
print("Mayuri Narsu Patil")
```

Mayuri Narsu Patil

Practical No. 5

Aim

To extract user data from a public REST API and a flat CSV file using Python, clean and transform the data, merge the datasets, and save the resulting data as a CSV file.

Practical NO. 5 : Extracting, Cleaning, and Merging Data from REST APIs and Flat Files Using Python

```
import pandas as pd
import requests
```

Add the API extraction function

```
def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()

        # Convert JSON data into a DataFrame
        return pd.json_normalize(data)

    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()
```

Add the CSV extraction function

```
def extract_csv_data(file_path):
    """Reads a flat CSV file."""
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()
```

Fetch data from the API

```
api_url = "https://sonplaceholder.typicode.com/users"

api_df = extract_api_data(api_url)

print("API Data:")
print(api_df)
```

```
API Data:
   id  name                username                email
0    1  Leanne Graham          Bret                Sincere@april.biz
1    2  Ervin Howell           Antonette          Shanna@melissa.tv
2    3  Clementine Baer             Samantha          Nathanael@vessel.net
3    4  Patricia Lebsack         Karianne          Julianne@conner.com
4    5  Charles Scriver            Kamren           Luke@mettling.com
5    6  Mrs. Dennis Schulist     Leopoldo Corkery  Karley@bachinger.info
6    7  Kurtis Weissnat           Elwyn Skiles      Telly.Hoeger@billy.biz
7    8  Nicholas Runolfsson      Maxime Hiennon    Sherwood@samuel.me
8    9  Glenna Reichert         Delphine          Chaim.McDermott@dana.io
9   10  Clementine DuBuque       Moriah Stanton    Rev.Fisher@karina.biz

   phone                website                address.street address.suite
0  1-770-736-8031 xs64a2d.org hildeganndale Victor Plains Suite 679
1  810-692-6593 x09125.net anastasia.info Victor Plains Suite 897
2  1-463-1-854-47 ramirez.info Douglas Extension Suite 847
3  493-170-9623 x156 kyle.biz Douglas Extension Suite 847
4  (254)954-1289 demarco.info Skiles walks Suite 351
5  1-477-9-6478 x0436e.com Norberto Crossing Suite 968
6  210-067-6132 elvis.io Rex Trail Suite 280
7  586-353-6943 x140 jacynthe.com Ellsworth Summit Suite 729
8  (775)976-6734 x0436e.com Devna Park Suite 449
9  024-648-3804 ambrose.net Kattie Turnpike Suite 198

   address.city address.zipcode address.geo.lat address.geo.lng
0  Gwenborough 92998-3874 -37.3159 81.1496
1  Wiskoyburgh 98566-7771 -43.9509 -34.4618
2  McKenziehaven 59590-4157 -68.6102 -67.0653
3  South Elviss 53919-4275 29.4572 -164.2990
4  Roscoeview 33263 -31.8129 62.5342
5  South Christy 23505-1337 97.4197 71.7478
6  Houmoutm 58804-1099 24.6918 21.8984
7  Allylview 45269 -14.2990 -120.7677
8  Bartholomewburg 76495-1109 -24.6463 18.8888
9  Lebsackbury 31428-2261 -38.2386 57.2232

   company.name company.catchPhrase
0  Romaguera-Crona Multi-layered client-server neural-net
1  Deckow-Crist Proactive didactic contingency
2  Romaguera-Jacobson Face to face bifurcated interface
3  Robel-Corkery Multi-tiered zero tolerance productivity
4  Keebler LLC User-centric fault-tolerant solution
5  Considine-Lockman Synchronized bottom-line interface
6  Johns Group Configurable multimedia task-force
7  Abernathy Group Implemented secondary concept
8  Yost and Sons Switchable context-sensitive mode
9  Hoeger LLC Centralized empowering task-force

   company.bs
0  harness real-time e-markets
1  synergize scalable supply-chains
2  e-enable strategic applications
3  transition cutting-edge services
4  revolutionize end-to-end systems
5  e-enable innovative applications
6  generate interactive e-tailers
7  e-enable extensible e-tailers
8  aggregate real-time technologies
9  transform end-to-end models
```

Select the required columns

```
api_df = api_df[["id", "name", "email", "company.name"]]
api_df.rename(columns={"company.name": "company"}, inplace=True)
print("Cleaned API Data:")
print(api_df)
```

Cleaned API Data:			
	id	name	email company
0	1	Leanne Graham	Sincere@april.biz Romaguera-Crona
1	2	Ervin Howell	Shanna@melissa.tv Deckow-Crist
2	3	Clementine Bauch	Mathias@vysenia.net Romaguera-Jacobson
3	4	Patricia Lebauc	Julianne.Donner@scry.com Rosen
4	5	Chelsey Dietrich	Lucio.Hettinger@annie.ca Keebler LLC
5	6	Mrs. Dennis Schulist	Karley.Dach@bispier.info Considine-Lockman
6	7	Kurtis Weissnat	Telly.Hoeger@biller.biz Johns Group
7	8	Nicholas Runolfsson Jr	Sherwood@scamond.me Abernathy Group
8	9	Glenna Reichert	Chaim.McDermott@yost.com Ward and Sons
9	10	Clementina DuBuque	Rey.Padberg@karina.biz Hoeger LLC

Create the flat CSV file

```
location_data = pd.DataFrame({
    "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "city": [
        "New York", "London", "Paris", "Tokyo", "Berlin",
        "Delhi", "Sydney", "Moscow", "Cairo", "Beijing"
    ],
    "country": [
        "USA", "UK", "France", "Japan", "Germany",
        "India", "Australia", "Russia", "Egypt", "China"
    ]
})
```

```

    ]
})

pd.DataFrame(location_data).to_csv("locations.csv", index=False)

print("locations.csv created successfully!")

locations.csv created successfully!

```

Read the CSV file

```

csv_df = extract_csv_data("locations.csv")

print("CSV Data:")
print(csv_df)

```

```

CSV Data:
   id  city  country
0   1  New York    USA
1   2  London     UK
2   3   Paris  France
3   4  Tokyo    Japan
4   5  Berlin  Germany
5   6   Delhi  India
6   7  Sydney  Australia
7   8  Moscow  Russia
8   9   Cairo  Egypt
9  10  Beijing  China

```

Merge API data and CSV data

```

merged_df = pd.merge(
    api_df,
    csv_df,
    on="id",
    how="inner"
)

print("\n--- Merged ETL Pipeline Data View ---")
print(merged_df)

```

```

--- Merged ETL Pipeline Data View ---
   id  name  email \
0   1  Leanne Graham  Sincere@april.biz
1   2  Ervin Howell  Shanna@melissa.tv
2   3  Clementine Bauch  Nathan@vesenia.net
3   4  Patricia Lebsack  Julianne.OConner@kory.org
4   5  Chelsey Dietrich  Lucio.Hettinger@annie.ca
5   6  Mrs. Dennis Schulist  Karley.Dach@jasper.info
6   7  Kurtis Weissnat  Telly.Hoeger@billy.biz
7   8  Nicholas Runolfsdottir V  Sherwood@rosamond.me
8   9  Glenna Reichert  Chaim.McDermott@dana.io
9  10  Clementina DuBuque  Rev.Padberg@karina.biz

   company  city  country
0  Romaguera-Crona  New York    USA
1  Deckow-Crist  London     UK
2  Romaguera-Jacobson  Paris  France
3  Robel-Corkery  Tokyo    Japan
4  Keebler LLC  Berlin  Germany
5  Considine-Lockman  Delhi  India
6  Johns Group  Sydney  Australia
7  Abernathy Group  Moscow  Russia
8  Yost and Sons  Cairo  Egypt
9  Hoeger LLC  Beijing  China

```

Save the final data

```

merged_df.to_csv(
    "cleaned_warehouse_profiles.csv",
    index=False
)

print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")

Data successfully saved to 'cleaned_warehouse_profiles.csv'

```

Download the final CSV

```

from google.colab import files

files.download("cleaned_warehouse_profiles.csv")

```

1. Extract

```

requests.get(api_url)

<Response [200]>

```

```

pd.read_csv("locations.csv")

```

```

   id  city  country
0   1  New York    USA
1   2  London     UK

```

2	3	Paris	France
3	4	Tokyo	Japan
4	5	Berlin	Germany
5	6	Delhi	India
6	7	Sydney	Australia
7	8	Moscow	Russia
8	9	Cairo	Egypt
9	10	Beijing	China

2. Transform

```
["id", "name", "email", "company.name"]
```

```
['id', 'name', 'email', 'company.name']
```

3. Load

```
pd.merge(api_df, csv_df, on="id", how="inner")
```

	id	name	email	company	city	country
0	1	Leanne Graham	Sincere@april.biz	Romaguera-Crona	New York	USA
1	2	Ervin Howell	Shanna@melissa.tv	Deckow-Crist	London	UK
2	3	Clementine Bauch	Nathan@yesenia.net	Romaguera-Jacobson	Paris	France
3	4	Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery	Tokyo	Japan
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Keebler LLC	Berlin	Germany
5	6	Mrs. Dennis Schulist	Karley_Dach@jasper.info	Considine-Lockman	Delhi	India
6	7	Kurtis Weissnat	Telly.Hoeger@billy.biz	Johns Group	Sydney	Australia
7	8	Nicholas Runolfsdottir V	Sherwood@rosamond.me	Abernathy Group	Moscow	Russia
8	9	Glenna Reichert	Chaim_McDermott@dana.io	Yost and Sons	Cairo	Egypt
9	10	Clementina DuBuque	Rey.Padberg@karina.biz	Hoeger LLC	Beijing	China

```
print("Mayuri Narsu Patil")
```

```
... Mayuri Narsu Patil
```

Practical No. 6

Aim

To configure a Directed Acyclic Graph (DAG) in Apache Airflow to run daily and execute sequential dependency tasks consisting of pipeline initialization, an extraction script, and pipeline success logging.

```
import airflow

print("Airflow version:", airflow.__version__)
```

Airflow version: 3.3.1

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="Lab assignment for API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )

    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="echo 'Running data extraction script...'",
    )

    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)"',
    )

    start_pipeline >> execute_extraction >> pipeline_complete

print("DAG created successfully!")
print("DAG ID:", dag.dag_id)
print("Tasks:", list(dag.task_ids))
```

DAG created successfully!
DAG ID: university_etl_orchestration
Tasks: ['start_pipeline', 'run_extraction_script', 'log_pipeline_success']

```
print("Mayuri Narsu Patil")
```

Mayuri Narsu Patil

Practical No. 7

Aim

To design and implement an ETL pipeline that extracts data from CSV and JSON files, cleans and transforms the data, identifies and removes invalid records, performs data validation, and loads the valid data into a relational database using incremental loading.

```
import pandas as pd
import sqlite3
import os

# 1. CREATE SAMPLE CSV AND JSON FILES
csv_file = "customers.csv"
json_file = "orders.json"
database_file = "etl_database.db"

# Sample CSV data
csv_data = pd.DataFrame({
    "CustomerID": [1, 2, 3, 4, 5, 6],
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Invalid"],
    "Email": [
        "alice@gmail.com",
        "bob@gmail.com",
        "charlie@gmail.com",
        "david@gmail.com",
        "eva@gmail.com",
        "invalid-email"
    ],
    "Age": [25, 32, 28, 40, 35, -5]
})
csv_data.to_csv(csv_file, index=False)

# Sample JSON data
json_data = [
    {"OrderID": 101, "CustomerID": 1, "Product": "Laptop", "Amount": 55000},
    {"OrderID": 102, "CustomerID": 2, "Product": "Mobile", "Amount": 25000},
    {"OrderID": 103, "CustomerID": 3, "Product": "Tablet", "Amount": 18000},
    {"OrderID": 104, "CustomerID": 4, "Product": "Headphones", "Amount": 3000},
    {"OrderID": 105, "CustomerID": 5, "Product": "Keyboard", "Amount": 2000},
    {"OrderID": 106, "CustomerID": 6, "Product": "Mouse", "Amount": -500}
]
pd.DataFrame(json_data).to_json(json_file, orient="records", indent=4)

# 2. EXTRACT DATA FROM CSV
customers = pd.read_csv(csv_file)

# Extract data from JSON
orders = pd.read_json(json_file)
print("CSV Data:")
print(customers)
print("\nJSON Data:")
print(orders)

# 3. CLEAN AND TRANSFORM CUSTOMER DATA

# Remove duplicate records
customers = customers.drop_duplicates()

# Remove extra spaces
customers["Name"] = customers["Name"].astype(str).str.strip()
```

```
customers["Email"] = customers["Email"].astype(str).str.strip().str.lower()
```

```
# Convert Age to numeric
```

```
customers["Age"] = pd.to_numeric(customers["Age"], errors="coerce")
```

```
# 4. CLEAN AND TRANSFORM ORDER DATA
```

```
orders = orders.drop_duplicates()
```

```
orders["Product"] = orders["Product"].astype(str).str.strip()
```

```
orders["CustomerID"] = pd.to_numeric(  
    orders["CustomerID"], errors="coerce"
```

```
)
```

```
orders["Amount"] = pd.to_numeric(  
    orders["Amount"], errors="coerce"
```

```
)
```

```
# 5. IDENTIFY INVALID CUSTOMER RECORDS
```

```
invalid_customers = customers[  
    (customers["CustomerID"].isna()) |  
    (customers["Name"].isna()) |  
    (customers["Email"].isna()) |  
    (~customers["Email"].str.contains("@", na=False)) |  
    (customers["Age"].isna()) |  
    (customers["Age"] <= 0)  
]
```

```
# Keep only valid customers
```

```
valid_customers = customers[  
    ~customers.index.isin(invalid_customers.index)  
].copy()
```

```
# 6. IDENTIFY INVALID ORDER RECORDS
```

```
invalid_orders = orders[  
    (orders["OrderID"].isna()) |  
    (orders["CustomerID"].isna()) |  
    (orders["Product"].isna()) |  
    (orders["Amount"].isna()) |  
    (orders["Amount"] <= 0)  
]
```

```
# Keep only valid orders
```

```
valid_orders = orders[  
    ~orders.index.isin(invalid_orders.index)  
].copy()
```

```
# 7. DATA VALIDATION
```

```
# Validate customer IDs
```

```
valid_customer_ids = set(valid_customers["CustomerID"])
```

```
# Orders must belong to an existing customer
```

```
valid_orders = valid_orders[  
    valid_orders["CustomerID"].isin(valid_customer_ids)  
].copy()
```

```
# Validate positive transaction amount
```

```
valid_orders = valid_orders[  
    valid_orders["Amount"] > 0  
].copy()
```

```
# 8. DISPLAY INVALID RECORDS
```

```
print("\nInvalid Customer Records Removed:")
```

```
print(invalid_customers)
```

```
print("\nInvalid Order Records Removed:")
```

```
print(invalid_orders)
```

```
# 9. CONNECT TO RELATIONAL DATABASE
```

```

conn = sqlite3.connect(database_file)

# 10. CREATE TABLES IF NOT EXISTS
conn.execute("""
CREATE TABLE IF NOT EXISTS customers (
    CustomerID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Email TEXT NOT NULL,
    Age INTEGER NOT NULL
)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS orders (
    OrderID INTEGER PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    Product TEXT NOT NULL,
    Amount REAL NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES customers(CustomerID)
)
""")

conn.commit()

# 11. INCREMENTAL LOADING - CUSTOMERS
existing_customers = pd.read_sql_query(
    "SELECT CustomerID FROM customers",
    conn
)

existing_customer_ids = set(
    existing_customers["CustomerID"].tolist()
)

new_customers = valid_customers[
    ~valid_customers["CustomerID"].isin(existing_customer_ids)
].copy()

if not new_customers.empty:
    new_customers.to_sql(
        "customers",
        conn,
        if_exists="append",
        index=False
    )

# 12. INCREMENTAL LOADING - ORDERS
existing_orders = pd.read_sql_query(
    "SELECT OrderID FROM orders",
    conn
)

existing_order_ids = set(
    existing_orders["OrderID"].tolist()
)

new_orders = valid_orders[
    ~valid_orders["OrderID"].isin(existing_order_ids)
].copy()

if not new_orders.empty:
    new_orders.to_sql(
        "orders",
        conn,
        if_exists="append",

```

```

        index=False
    )
# 13. FINAL DATA VALIDATION IN DATABASE
final_customers = pd.read_sql_query(
    "SELECT * FROM customers",
    conn
)
final_orders = pd.read_sql_query(
    "SELECT * FROM orders",
    conn
)
# Check for invalid database records
invalid_db_customers = final_customers[
    (final_customers["Age"] <= 0) |
    (~final_customers["Email"].str.contains("@", na=False))
]
invalid_db_orders = final_orders[
    (final_orders["Amount"] <= 0)
]
# 14. DISPLAY FINAL RESULTS
print("\n===== ETL PIPELINE RESULT =====")
print("\nValid Customers Loaded:")
print(final_customers)
print("\nValid Orders Loaded:")
print(final_orders)
print("\nNumber of Invalid Customers Removed:",
      len(invalid_customers))
print("Number of Invalid Orders Removed:",
      len(invalid_orders))
print("New Customers Loaded:",
      len(new_customers))
print("New Orders Loaded:",
      len(new_orders))
print("Invalid Records in Database:",
      len(invalid_db_customers) + len(invalid_db_orders))
print("\nETL Pipeline Completed Successfully!")
# 15. CLOSE DATABASE CONNECTION
conn.close()

```

```

--- CSV Data:
  CustomerID  Name  Email  Age
0           1  Alice  alice@email.com  25
1           2   Bob  bob@email.com  32
2           3 Charlie  charlie@email.com  28
3           4  David  david@email.com  40
4           5   Eva  eva@email.com  35
5           6 Invalid  invalid-email  -5

JSON Data:
  OrderID  CustomerID  Product  Amount
0       101           1  Laptop   55000
1       102           2  Mobile   25000
2       103           3  Tablet   18000
3       104           4  Headphones  3000
4       105           5  Keyboard   2000
5       106           6  Mouse     -500

Invalid Customer Records Removed:
  CustomerID  Name  Email  Age
5           6 Invalid  invalid-email  -5

Invalid Order Records Removed:
  OrderID  CustomerID  Product  Amount
5       106           6  Mouse     -500

===== ETL PIPELINE RESULT =====
Valid Customers Loaded:
  CustomerID  Name  Email  Age
0           1  Alice  alice@email.com  25
1           2   Bob  bob@email.com  32
2           3 Charlie  charlie@email.com  28
3           4  David  david@email.com  40
4           5   Eva  eva@email.com  35

Valid Orders Loaded:
  OrderID  CustomerID  Product  Amount
0       101           1  Laptop   55000.0
1       102           2  Mobile   25000.0
2       103           3  Tablet   18000.0
3       104           4  Headphones  3000.0
4       105           5  Keyboard   2000.0

Number of Invalid Customers Removed: 1
Number of Invalid Orders Removed: 1
New Customers Loaded: 5
New Orders Loaded: 5
Invalid Records in Database: 0

ETL Pipeline Completed Successfully!

print("Mayuri Narsu Patil")
Mayuri Narsu Patil

```

Practical No. 8

Aim

To implement PySpark programs for reading a CSV dataset, displaying its schema, performing filtering, grouping and aggregation, removing duplicate records, joining two datasets, and calculating average sales by product category.

Code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum, count

# Create Spark session
spark = SparkSession.builder \
    .appName("Practical_8_PySpark") \
    .getOrCreate()

# Create sample datasets
sales_data = [
    (1, "Laptop", "Electronics", 55000),
    (2, "Mobile", "Electronics", 25000),
    (3, "Shirt", "Clothing", 1500),
    (4, "Jeans", "Clothing", 2500),
    (5, "Tablet", "Electronics", 18000),
    (6, "Shoes", "Footwear", 3000),
    (7, "Headphones", "Electronics", 2000),
    (8, "Jacket", "Clothing", 3500),
    (9, "Watch", "Accessories", 4000),
    (10, "Bag", "Accessories", 2200),
    (3, "Shirt", "Clothing", 1500)
]

sales_columns = ["SaleID", "Product", "Category", "Sales"]
sales_df = spark.createDataFrame(sales_data, sales_columns)

customer_data = [
    (1, "Customer A"),
    (2, "Customer B"),
    (3, "Customer C"),
    (4, "Customer D"),
    (5, "Customer E"),
    (6, "Customer F"),
    (7, "Customer G"),
    (8, "Customer H"),
    (9, "Customer I"),
    (10, "Customer J")
]

customer_columns = ["SaleID", "CustomerName"]
customers_df = spark.createDataFrame(customer_data, customer_columns)

# Write and read CSV dataset
csv_path = "sales_data.csv"
sales_df.write.mode("overwrite").option("header", True).csv(csv_path)
df = spark.read.option("header", True).option("inferSchema", True).csv(csv_path)
print("CSV Dataset:")
df.show()

# Display schema
print("Schema:")
df.printSchema()

# Filtering
print("Filtered Sales above 5000:")
df.filter(col("Sales") > 5000).show()
```

Grouping and aggregation

```
print("Total Sales by Category:")
df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales"),
    count("SaleID").alias("Number_of_Sales")
).show()
```

Remove duplicate records

```
df_unique = df.dropDuplicates()
print("Dataset after removing duplicates:")
df_unique.show()
```

Join two datasets

```
joined_df = df_unique.join(customers_df, on="SaleID", how="inner")
print("Joined Dataset:")
joined_df.show()
```

Average sales by product category

```
print("Average Sales by Product Category:")
average_sales = df_unique.groupBy("Category").agg(
    avg("Sales").alias("Average_Sales")
)
average_sales.show()
```

Stop Spark session

```
spark.stop()
```

CSV Dataset:

```
***
+-----+-----+-----+-----+
|SaleID|Product|Category|Sales|
+-----+-----+-----+-----+
|6|Shoes|Footwear|3000|
|7|Headphones|Electronics|2000|
|8|Jacket|Clothing|3500|
|9|Watch|Accessories|4000|
|10|Bag|Accessories|2200|
|3|Shirt|Clothing|1500|
|1|Laptop|Electronics|55000|
|2|Mobile|Electronics|25000|
|3|Shirt|Clothing|1500|
|4|Jeans|Clothing|2500|
|5|Tablet|Electronics|18000|
+-----+-----+-----+-----+
```

Schema:

```
root
|-- SaleID: integer (nullable = true)
|-- Product: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sales: integer (nullable = true)
```

Filtered Sales above 5000:

```
+-----+-----+-----+-----+
|SaleID|Product|Category|Sales|
+-----+-----+-----+-----+
|1|Laptop|Electronics|55000|
|2|Mobile|Electronics|25000|
|5|Tablet|Electronics|18000|
+-----+-----+-----+-----+
```

Total Sales by Category:

```
***
+-----+-----+-----+
|Category|Total_Sales|Number_of_Sales|
+-----+-----+-----+
|Electronics|100000|4|
|Clothing|9000|4|
|Footwear|3000|1|
|Accessories|6200|2|
+-----+-----+-----+
```

Dataset after removing duplicates:

```
+-----+-----+-----+-----+
|SaleID|Product|Category|Sales|
+-----+-----+-----+-----+
|8|Jacket|Clothing|3500|
|6|Shoes|Footwear|3000|
|3|Shirt|Clothing|1500|
|9|Watch|Accessories|4000|
|10|Bag|Accessories|2200|
|7|Headphones|Electronics|2000|
|1|Laptop|Electronics|55000|
|5|Tablet|Electronics|18000|
|4|Jeans|Clothing|2500|
|2|Mobile|Electronics|25000|
+-----+-----+-----+-----+
```

Joined Dataset:

```
+-----+-----+-----+-----+-----+
|SaleID|Product|Category|Sales|CustomerName|
+-----+-----+-----+-----+-----+
|1|Laptop|Electronics|55000|Customer A|
|2|Mobile|Electronics|25000|Customer B|
|3|Shirt|Clothing|1500|Customer C|
|4|Jeans|Clothing|2500|Customer D|
|5|Tablet|Electronics|18000|Customer E|
|6|Shoes|Footwear|3000|Customer F|
|7|Headphones|Electronics|2000|Customer G|
|8|Jacket|Clothing|3500|Customer H|
|9|Watch|Accessories|4000|Customer I|
|10|Bag|Accessories|2200|Customer J|
+-----+-----+-----+-----+-----+
```

Average Sales by Product Category:

```
+-----+-----+
|Category|Average_Sales|
+-----+-----+
|Electronics|25000.0|
|Clothing|2500.0|
|Footwear|3000.0|
|Accessories|3100.0|
+-----+-----+
```

```
print("Name: Mayuri Narsu Patil")
```

```
Name: Mayuri Narsu Patil
```

Practical No. 9

Aim

Build an end-to-end pipeline: CSV → Python/Pandas → Data Cleaning → SQL Database → Report. Build an ETL pipeline for an e-commerce dataset containing customers, orders, products, and payments. Design a data pipeline that handles both historical and newly arriving data.

Code:

```
import pandas as pd
import sqlite3
import os

# Create sample e-commerce CSV files
customers_data = pd.DataFrame({
    "CustomerID": [1, 2, 3, 4, 5],
    "CustomerName": ["Customer A", "Customer B", "Customer C", "Customer D", "Customer E"],
    "Email": ["a@gmail.com", "b@gmail.com", "c@gmail.com", "d@gmail.com", "e@gmail.com"],
    "City": ["Mumbai", "Pune", "Nashik", "Thane", "Kalyan"]
})

products_data = pd.DataFrame({
    "ProductID": [101, 102, 103, 104, 105],
    "ProductName": ["Laptop", "Mobile", "Headphones", "Shoes", "Bag"],
    "Category": ["Electronics", "Electronics", "Electronics", "Fashion", "Fashion"],
    "Price": [55000, 25000, 2000, 3000, 2200]
})

orders_data = pd.DataFrame({
    "OrderID": [1001, 1002, 1003, 1004, 1005],
    "CustomerID": [1, 2, 3, 4, 5],
    "ProductID": [101, 102, 103, 104, 105],
    "Quantity": [1, 2, 3, 1, 2],
    "OrderDate": ["2026-01-10", "2026-02-15", "2026-03-20", "2026-04-12", "2026-05-18"]
})

payments_data = pd.DataFrame({
    "PaymentID": [501, 502, 503, 504, 505],
    "OrderID": [1001, 1002, 1003, 1004, 1005],
    "PaymentMethod": ["UPI", "Card", "Cash", "UPI", "Card"],
    "PaymentStatus": ["Paid", "Paid", "Paid", "Paid", "Pending"]
})

customers_data.to_csv("customers.csv", index=False)
products_data.to_csv("products.csv", index=False)
orders_data.to_csv("orders.csv", index=False)
payments_data.to_csv("payments.csv", index=False)

# Extract CSV data
customers = pd.read_csv("customers.csv")
products = pd.read_csv("products.csv")
orders = pd.read_csv("orders.csv")
payments = pd.read_csv("payments.csv")

# Data cleaning
customers = customers.drop_duplicates()
products = products.drop_duplicates()
orders = orders.drop_duplicates()
payments = payments.drop_duplicates()
```

```
customers["CustomerName"] = customers["CustomerName"].str.strip()
customers["Email"] = customers["Email"].str.strip().str.lower()

products["ProductName"] = products["ProductName"].str.strip()
products["Price"] = pd.to_numeric(products["Price"], errors="coerce")

orders["Quantity"] = pd.to_numeric(orders["Quantity"], errors="coerce")
orders["OrderDate"] = pd.to_datetime(orders["OrderDate"], errors="coerce")
```

```
payments["PaymentMethod"] = payments["PaymentMethod"].str.strip()
payments["PaymentStatus"] = payments["PaymentStatus"].str.strip()
```

```
# Remove invalid records
customers = customers.dropna()
products = products.dropna()
orders = orders.dropna()
payments = payments.dropna()
```

```
orders = orders[orders["Quantity"] > 0]
products = products[products["Price"] > 0]
```

```
# Create SQL database
conn = sqlite3.connect("ecommerce.db")
```

```
# Historical data loading
customers.to_sql("customers", conn, if_exists="replace", index=False)
products.to_sql("products", conn, if_exists="replace", index=False)
orders.to_sql("orders", conn, if_exists="replace", index=False)
payments.to_sql("payments", conn, if_exists="replace", index=False)
```

```
# Newly arriving data
new_orders = pd.DataFrame({
    "OrderID": [1006, 1007],
    "CustomerID": [1, 3],
    "ProductID": [102, 101],
    "Quantity": [1, 1],
    "OrderDate": ["2026-06-10", "2026-06-15"]
})
new_orders["OrderDate"] = pd.to_datetime(new_orders["OrderDate"])
new_orders["Quantity"] = pd.to_numeric(new_orders["Quantity"])

existing_orders = pd.read_sql("SELECT OrderID FROM orders", conn)

new_orders = new_orders[
    ~new_orders["OrderID"].isin(existing_orders["OrderID"])
]
```

```
new_orders.to_sql("orders", conn, if_exists="append", index=False)
```

```
# Generate report
report = pd.read_sql("""
SELECT
    o.OrderID,
    c.CustomerName,
    p.ProductName,
    p.Category,
    p.Price,
    o.Quantity,
```

```

(p.Price * o.Quantity) AS TotalAmount,
o.OrderDate,
pay.PaymentMethod,
pay.PaymentStatus
FROM orders o
JOIN customers c ON o.CustomerID = c.CustomerID
JOIN products p ON o.ProductID = p.ProductID
LEFT JOIN payments pay ON o.OrderID = pay.OrderID
ORDER BY o.OrderDate
""" , conn)

```

```

print("E-COMMERCE SALES REPORT")
print(report)

```

```

print("\nSUMMARY REPORT")
print("Total Customers:", len(customers))
print("Total Products:", len(products))
print("Total Orders:", len(report))
print("Total Sales:", report["TotalAmount"].sum())

```

```

print("\nSales by Category:")
print(
    report.groupby("Category")["TotalAmount"]
    .sum()
    .reset_index()
    .rename(columns={"TotalAmount": "TotalSales"})
)
print("\nPayment Status:")
print(report["PaymentStatus"].value_counts())
conn.close()
print("\nETL Pipeline Completed Successfully!")

```

```

E-COMMERCE SALES REPORT
***
  OrderID  CustomerName  ProductName  Category  Price  Quantity  \
0      1001    Customer A      Laptop  Electronics  55000         1
1      1002    Customer B      Mobile  Electronics  25000         2
2      1003    Customer C  Headphones  Electronics   2000         3
3      1004    Customer D       Shoes    Fashion    3000         1
4      1005    Customer E        Bag    Fashion    2200         2
5      1006    Customer A      Mobile  Electronics  25000         1
6      1007    Customer C      Laptop  Electronics  55000         1

   TotalAmount  OrderDate  PaymentMethod  PaymentStatus
0      55000  2026-01-10 00:00:00         UPI          Paid
1      50000  2026-02-15 00:00:00         Card          Paid
2       6000  2026-03-20 00:00:00         Cash          Paid
3       3000  2026-04-12 00:00:00         UPI          Paid
4       4400  2026-05-18 00:00:00         Card    Pending
5      25000  2026-06-10 00:00:00        None          None
6      55000  2026-06-15 00:00:00        None          None

SUMMARY REPORT
Total Customers: 5
Total Products: 5
Total Orders: 7
Total Sales: 198400

Sales by Category:
   Category  TotalSales
0  Electronics    191000
1    Fashion       7400

Payment Status:
PaymentStatus
Paid      4
Pending   1
Name: count, dtype: int64

ETL Pipeline Completed Successfully!

```

Practical No. 10

Aim

Mini Project: Design and implement an end-to-end data engineering solution including data ingestion, cleaning, transformation, storage, data warehouse design, and reporting.

1. Data Ingestion

```
All datasets loaded successfully!
Products: (77, 10)
Orders: (830, 14)
Order Details: (2155, 5)
```

2. Data Cleaning

```
# Check missing values
print("Missing Values:")
print("Products:\n", products.isnull().sum())
print("\nOrders:\n", orders.isnull().sum())
print("\nOrder Details:\n", order_details.isnull().sum())

# Check duplicate records
print("\nDuplicate Records:")
print("Products:", products.duplicated().sum())
print("Orders:", orders.duplicated().sum())
print("Order Details:", order_details.duplicated().sum())

... Missing Values:
Products:
ProductID      0
ProductName     0
SupplierID     0
CategoryID     0
QuantityPerUnit 0
UnitPrice      0
UnitsInStock   0
UnitsOnOrder   0
ReorderLevel   0
Discontinued   0
dtype: int64

Orders:
OrderID      0
CustomerID   0
EmployeeID   0
OrderDate    0
RequiredDate 0
ShippedDate  21
ShipVia      0
Freight      0
ShipName     0
ShipAddress  0
ShipCity     0
ShipRegion   507
ShipPostalCode 19
ShipCountry  0
dtype: int64

Order Details:
OrderID      0
ProductID    0
UnitPrice    0
Quantity     0
Discount     0
dtype: int64

Duplicate Records:
Products: 0
Orders: 0
Order Details: 0
```

3. Data Transformation

All three datasets merged successfully!
Rows and columns: (2155, 27)

OrderID	ProductID	UnitPrice_x	Quantity	Discount	CustomerID	EmployeeID	OrderDate	RequiredDate	ShippedDate	...	ShipCountry	ProductName	SupplierID	CategoryID	QuantityPerUnit	UnitPrice_y	UnitsInStock
0	10248	11	14.0	12	0	VINET	5	1996-07-04	1996-08-01 00:00:00	1996-07-16 00:00:00	...	France	Queso Cabrales	5	4	1 kg pkg.	21.00
1	10248	42	9.8	10	0	VINET	5	1996-07-04	1996-08-01 00:00:00	1996-07-16 00:00:00	...	France	Singaporean Hokkien Fried Mee	20	5	32 - 1 kg pkgs.	14.00
2	10248	72	34.8	5	0	VINET	5	1996-07-04	1996-08-01 00:00:00	1996-07-16 00:00:00	...	France	Mozzarella di Giovanni	14	4	24 - 200 g pkgs.	34.80
3	10249	14	18.6	9	0	TOMSP	6	1996-07-05	1996-08-16 00:00:00	1996-07-10 00:00:00	...	Germany	Tofu	6	7	40 - 100 g pkgs.	23.25
4	10249	51	42.4	40	0	TOMSP	6	1996-07-05	1996-08-16 00:00:00	1996-07-10 00:00:00	...	Germany	Manjimp Dried Apples	24	7	50 - 300 g pkgs.	53.00

5 rows x 27 columns

Sales Amount calculated successfully!

OrderID	ProductID	Quantity	UnitPrice_x	Discount	SalesAmount
0	10248	11	14.0	0	168.0
1	10248	42	9.8	0	98.0
2	10248	72	34.8	0	174.0
3	10249	14	18.6	0	167.4
4	10249	51	42.4	0	1696.0

4. Data Storage

```
print("File: transformed_sales_data.csv")

Transformed data stored successfully!
File: transformed_sales_data.csv
```

5. Data Warehouse Design

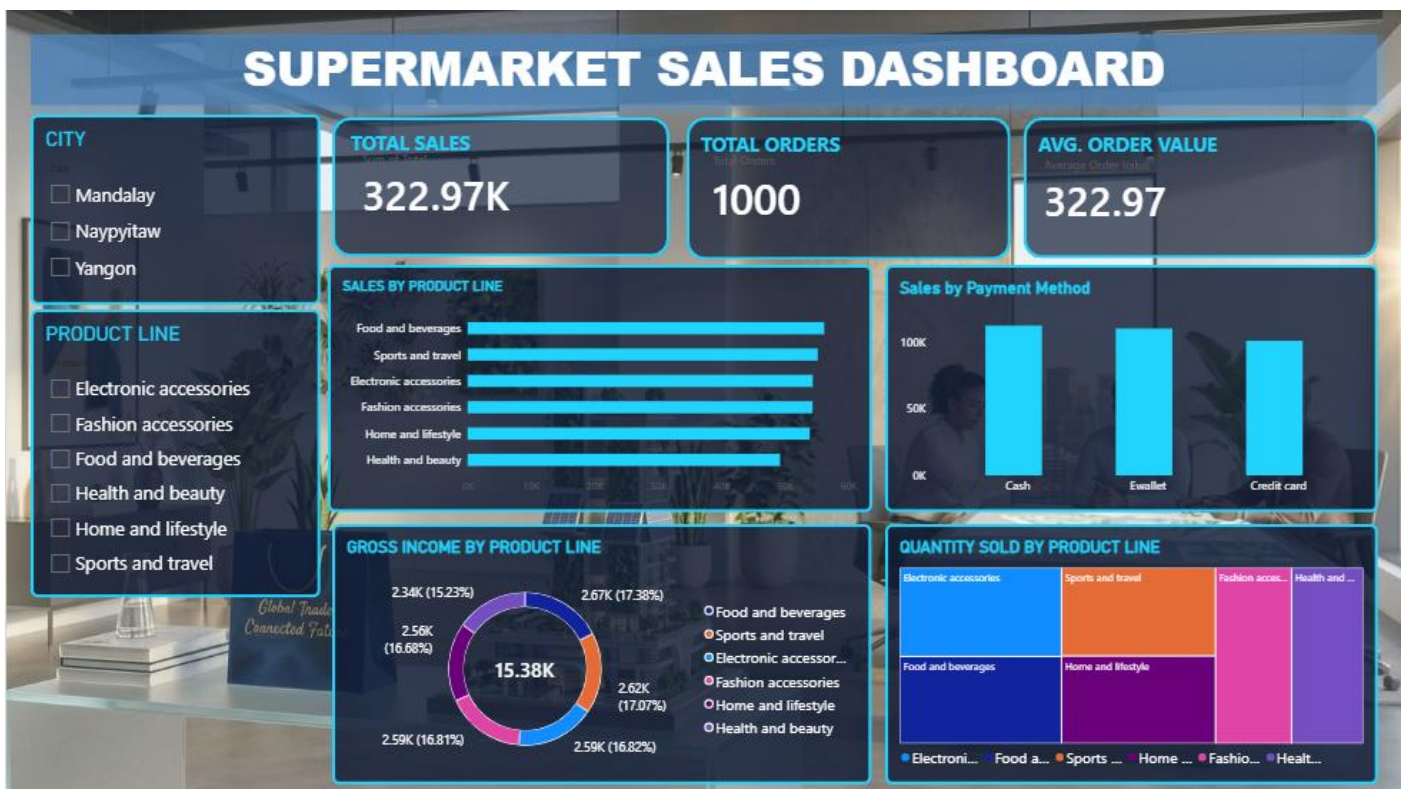
Dimension tables created successfully!
Dim Product: (77, 6)
Dim Order: (830, 6)

FactSales table created successfully!
Rows and Columns: (2155, 6)

	OrderID	ProductID	Quantity	UnitPrice_x	Discount	SalesAmount
0	10248	11	12	14.0	0	168.0
1	10248	42	10	9.8	0	98.0
2	10248	72	5	34.8	0	174.0
3	10249	14	9	18.6	0	167.4
4	10249	51	40	42.4	0	1696.0

*** Data Warehouse tables stored successfully!
DimProduct.csv
DimOrder.csv
FactSales.csv

6. Reporting / Visualization



7. Conclusion

The project successfully implemented an end-to-end data engineering pipeline using multiple CSV datasets. The data was ingested, cleaned, transformed, integrated, stored, and organized into a star-schema-based data warehouse. The processed data was then used for reporting and visualization through Power BI.